



INSTITUTO FEDERAL
Rio Grande do Sul

Banco de Dados II

Armazenamento e indexação

Prof. Edimar Manica

edimar.manica@ibiruba.ifrs.edu.br



Referência

- Livro
 - Título: Database Management Systems
 - Autores: Ramakrishnan and J. Gehrke
 - Edição: 3ª

Armazenamento externo de dados

- **Discos:**
 - Recuperam blocos randômicos a um custo fixo
 - Porém, a leitura de blocos consecutivos é muito mais rápido que a leitura de blocos em ordem aleatória
- **Fitas:**
 - Podem ler blocos em ordem sequencial
 - Mais baratos que discos; são usados para arquivar dados
- **Organização de Arquivos:** é um método para organizar registros de arquivos em um dispositivo de armazenamento externo.
 - **Id de Registro (rid):** é utilizado para localizar um registro
 - **Índices:** são estruturas de dados que permitem achar o **Id** de registros que possuem um determinado valor para uma chave de pesquisa
- **Arquitetura:** O gerenciador de *buffer* executa a transferência de blocos do armazenamento externo para a memória principal

Formas de organização de arquivos

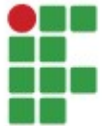
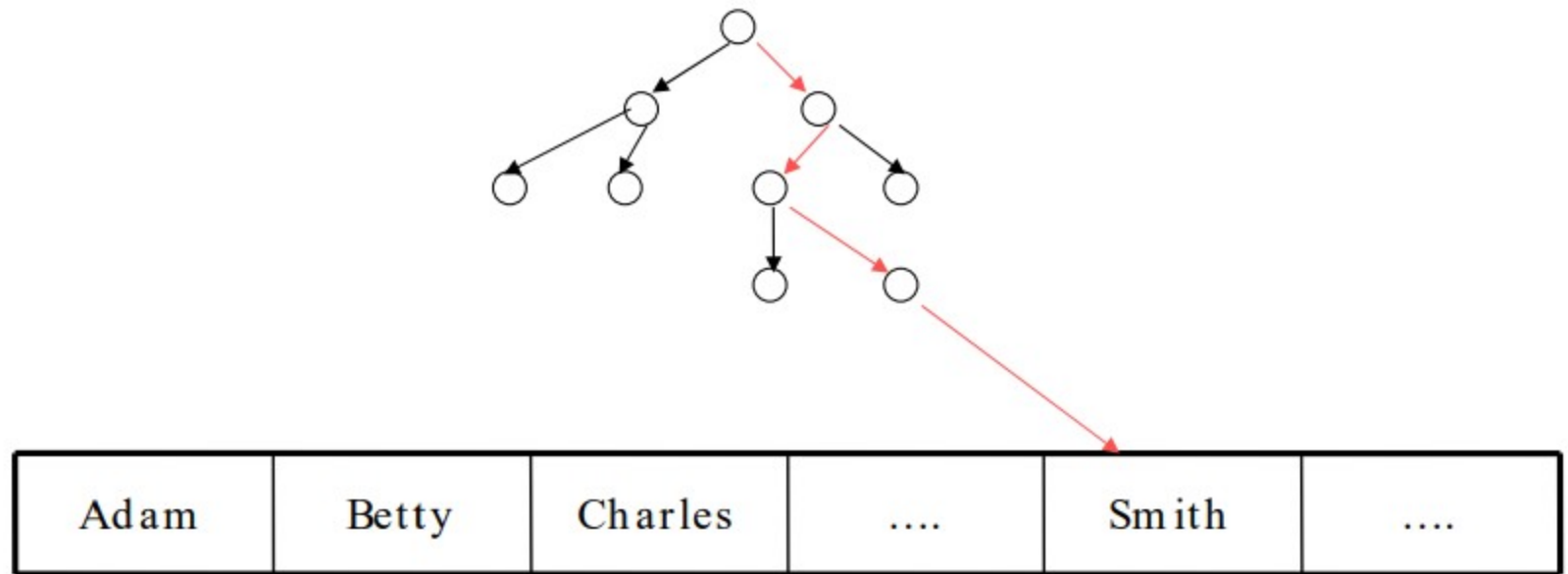
- Existem diversas alternativas, que são apropriadas para algumas situações e não tão boas para outras:
 - **Arquivos Heap** (ordem randômica): apropriado quando o acesso típico é a busca de todos os registros
 - **Arquivos Ordenados**: recomendados quando os registros devem ser recuperados em alguma ordem, ou quando somente registros em um intervalo de valores são necessários
 - **Índices**: estruturas de dados que organizam os registros utilizando uma árvore ou função de hash
 - Similar aos arquivos ordenados, os índices são recomendados para a obtenção de registros dentro de um intervalo de valores da “chave de pesquisa”
 - Atualizações são muito rápidas, comparadas a arquivos ordenados

Índices

- São muito importantes para melhorar o tempo de processamento de uma consulta
- Considere a relação:
 - `Pessoa(codigo, nome, nascimento, cod_cid#)`
- Considere a consulta:
 - `SELECT * FROM pessoa WHERE nome='Joana'`
- O processamento sequencial do arquivo **Pessoa** pode levar muito tempo.

Índices

- Criação de um índice sobre o atributo nome:



Índices

- Um **índice** sobre um arquivo agiliza a seleção de registros baseada nos **campos da chave de pesquisa** utilizados para criar o índice.
 - Qualquer atributo de uma relação pode fazer parte dos campos da chave de pesquisa do índice.
 - **Chave de pesquisa** não é o mesmo que **chave** (conjunto mínimo de atributos que identificam unicamente uma tupla da relação).

Índices

- Um índice contém uma coleção de **entradas de dados**, cada uma com um valor de chave k (representado por k^*)
 - Na **entrada de dados** k^* podemos armazenar:
 - 1) O registro de dados com chave de busca k
 - 2) $\langle k, \text{rid do registro de dado com chave de busca } k \rangle$
 - 3) $\langle k, \text{lista de rids dos registros com chave de busca } k \rangle$
 - Dada uma entrada de dados k^* , é possível recuperar o registro de dados com chave k com no máximo um acesso a disco.

Analizando as alternativas de entrada de dados

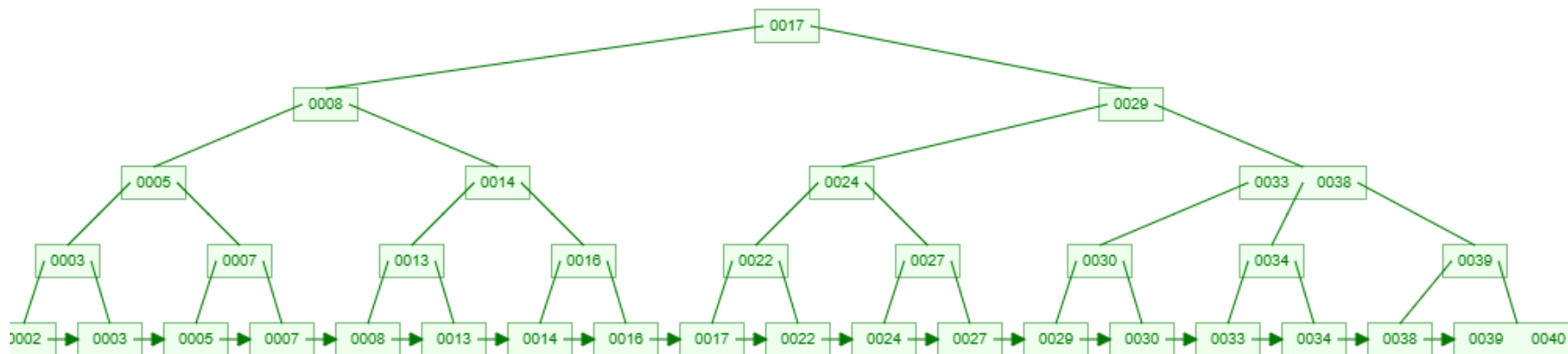
- **Alternativa 01:** o registro de dados com chave de busca k
 - Neste caso, a própria estrutura de índice corresponde a organização de arquivos utilizada para armazenar os registros de dados
 - No máximo um índice pode utilizar essa alternativa para um determinado arquivo (tabela). Caso contrário, haverá duplicação de registros, que tem como consequência redundância de armazenamento e pode causar inconsistências)
 - Se o tamanho do registro de dados é grande, o número de blocos contendo entradas de dados é grande. Isto implica que o tamanho das informações auxiliares no índice também pode ser grande

Analizando as alternativas de entrada de dados

- **Alternativa 02:** $\langle k, \text{rid do registro de dado com chave de busca } k \rangle$
- **Alternativa 03:** $\langle k, \text{lista de rids dos registros com chave de busca } k \rangle$
- **Análise**
 - Nas alternativas 02 e 03, as entradas de dados em geral são bem menores que os registros de dados. Portanto, esta alternativa é melhor se os registros de dados são grandes, principalmente quando a chave de pesquisa é pequena
 - A Alternativa 3 é mais compacta que a Alternativa 2, mas utiliza entradas de dados de tamanho variável, mesmo quando a chave de pesquisa é de tamanho fixo

Índices com árvores B+

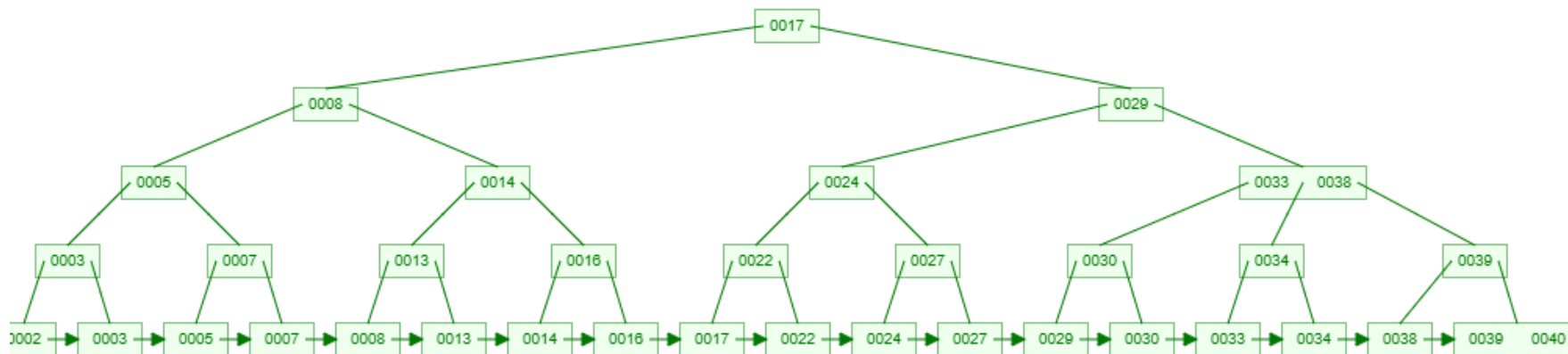
- Exemplo de árvore B+ de grau 3



- Os **nodos folha** contém todas as chaves de pesquisa e as entradas de dados. Além disso, eles são encadeados (anterior, próximo)
- Os **nodos internos** contém apenas as chaves de pesquisa (usados somente para direcionar a busca)
- Simulação: <https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

Índices com árvores B+

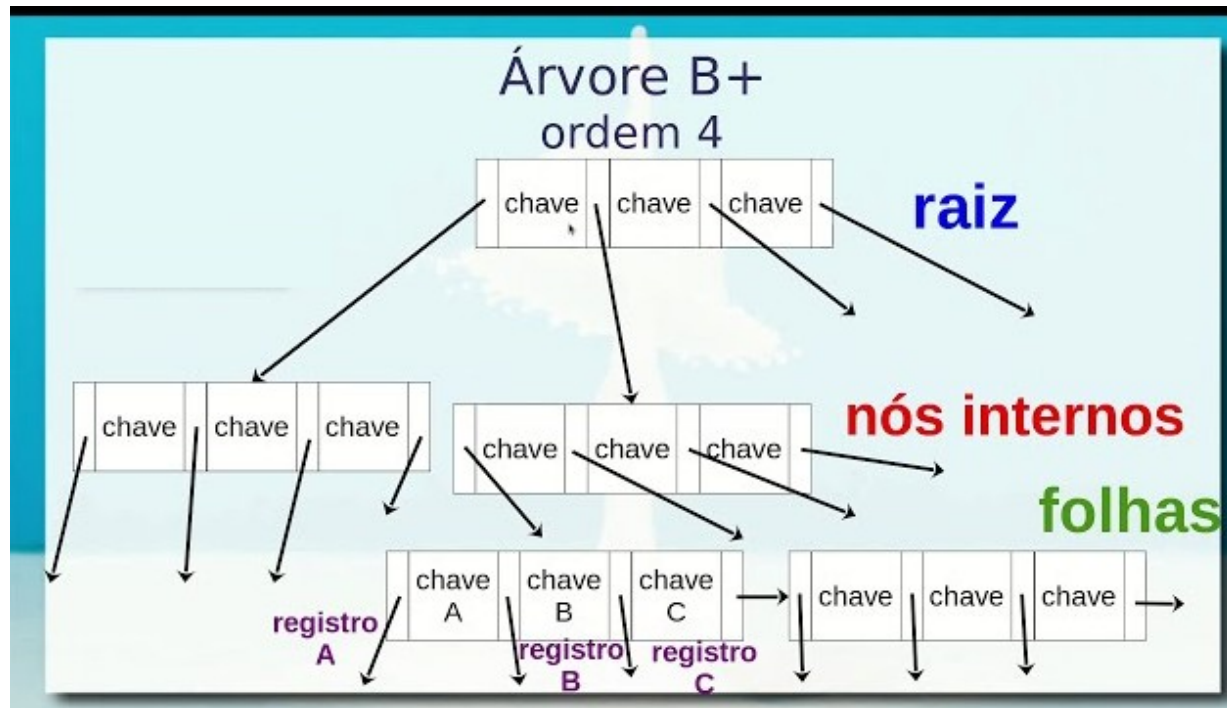
- Exemplo de árvore B+ de grau 3



- Encontre os seguintes registros:
 - 27
 - 29
 - Todos > 15
 - Todos < 30

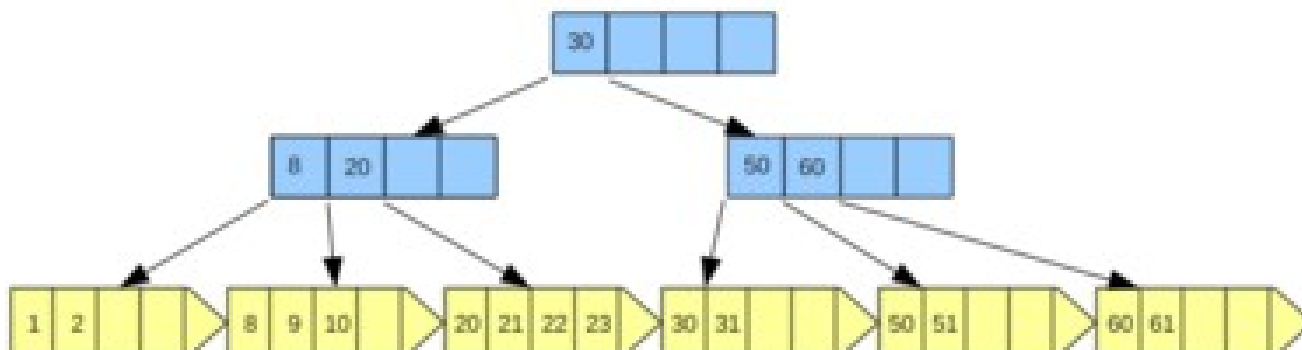
Índices com árvores B+

- Exemplo de árvore B+ de grau 4 (número máximo de filhos de um nó)



Índices com árvores B+

- Exemplo de árvore B+ de grau 4 (número máximo de filhos de um nó)



Alguns nós podem ficar vazios para facilitar inserções futuras.

Índices com árvores B+

- São melhores para testes de intervalo
- Também são bons para teste de igualdade



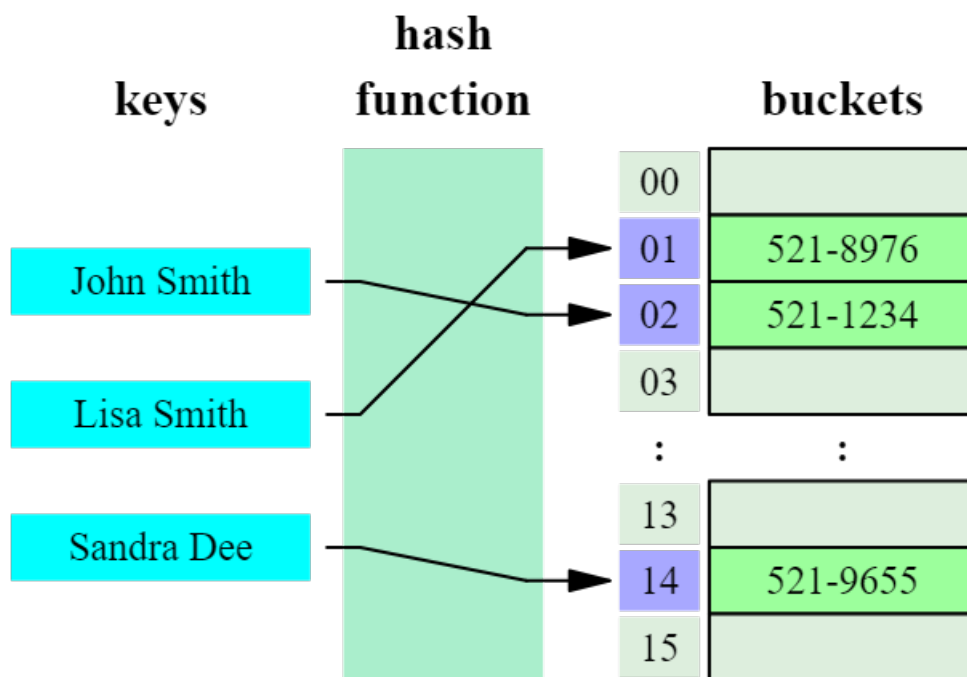
Índices baseados em Hash

- Bom para seleções que utilizam o teste de igualdade
- O índice é uma coleção de **buckets**
 - Um **Bucket** = um bloco primário mais zero ou mais blocos de *overflow*
 - Os buckets contêm ponteiros para os registros
- **Função de Hash**: bucket no qual a entrada de dados do registro *r* deve estar contida
 - Não há necessidade de “entradas de índice” nesta estrutura de dados



Índices baseados em Hash

- Exemplo de função de hash



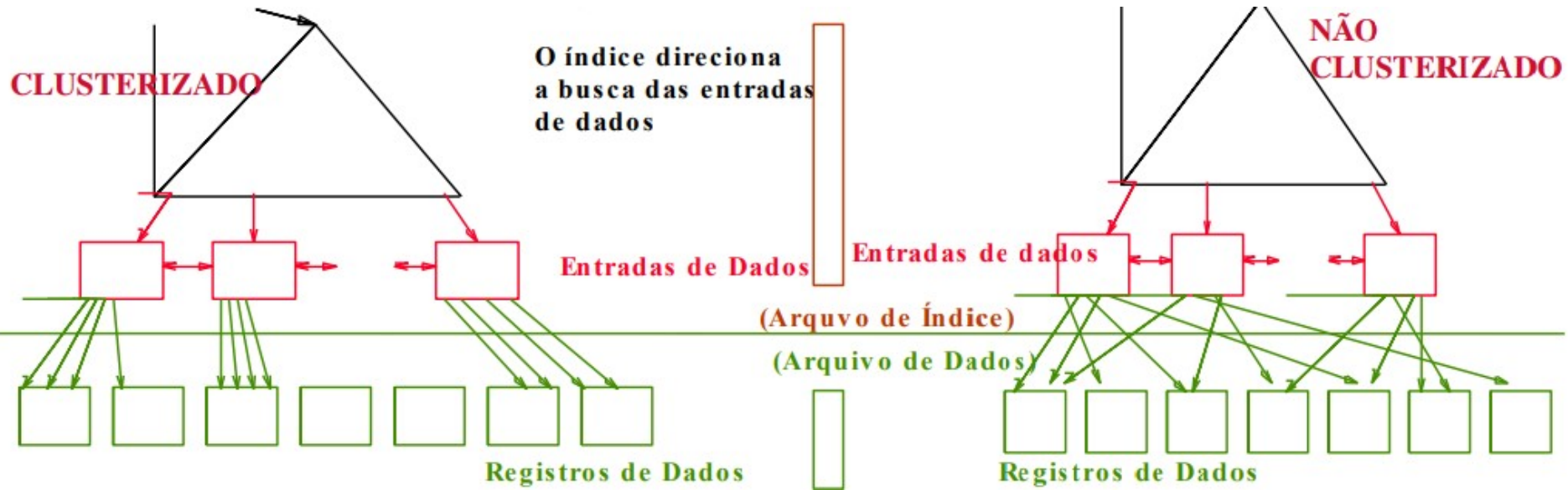
Classificação dos índices

- **Primário vs. secundário:** se a chave de pesquisa é uma chave primária, então o índice é primário.
 - Índice **Unique**: a chave de pesquisa contém uma chave candidata
- **Clusterizado vs. não clusterizado:** se a ordem dos registros de dados for igual ou próxima a ordem das entradas de dados do índice, então o índice é clusterizado
 - A **alternativa 1** de entrada de dados implica que o índice é clusterizado
 - Um arquivo pode estar clusterizado no máximo por uma chave de pesquisa.
 - A clusterização pode afetar MUITO o custo de recuperação dos registros de dados através dos índices



Índices clusterizados

- Suponha que a **Alternativa 2** seja usada para as entradas de dados e os registros estão armazenados em um arquivo “Heap” (desordenado).
 - Para criar um índice clusterizado, ordena-se o arquivo de dados (com espaço livre para inserções futuras).
 - Nós de overflow podem ser necessários para inserções (portanto, a ordem dos registros é próxima, mas não idêntica a das entradas de dados)



Analizando a carga de trabalho

- Para cada consulta, analisar:
 - ✓ Quais relações são utilizadas?
 - ✓ Quais os atributos retornados pela consulta?
 - ✓ Quais os atributos estão nas condições de junção e seleção?
 - ✓ Quão seletivas são estas condições?
- Para cada atualização, analisar:
 - ✓ Quais atributos estão nas condições de seleção e junção? Quão seletivas são estas condições?
 - ✓ O tipo de atualização (INSERT/DELETE/UPDATE), e os atributos que são atualizados



A escolha dos índices

- Quantos índices criar?
 - ✓ Quais as relações que devem ter índices?
 - ✓ Quais os atributos que devem formar a chave de pesquisa?
 - ✓ Devemos criar diversos índices?
- Para cada índice, que tipo de índice ele deve ser?
 - ✓ Clusterizado?
 - ✓ Hash ou árvore?



A escolha dos índices

- Considere as consultas mais importantes e os planos de consulta utilizando os índices existentes. Verifique se é possível obter um plano de consulta melhor se existirem índices adicionais. Em caso afirmativo, crie novos índices.
 - ✓ É necessário entender como um SGBD avalia consultas e cria planos de avaliação de consultas.
- Antes de criar um índice, deve ser avaliado também o impacto da sua criação nas atualizações.
 - ✓ **Trade-off:** os índices podem tornar as consultas mais rápidas e atualizações mais lentas. Eles também requerem espaço em disco



Guia para seleção de índices

- Atributos na cláusula WHERE são candidatos a chaves de pesquisa de um índice
 - ✓ **Teste de igualdade:**
 - Ex: idade = 30
 - Recomendação: tabela hash
 - ✓ **Teste de intervalo:**
 - Ex: idade entre 30 e 40
 - Recomendação: árvore B+
 - ✓ A clusterização ajuda nas consultas envolvendo teste de intervalo; também ajuda quando o teste é de igualdade e houver valores duplicados.



Guia para seleção de índices

- Chaves de pesquisa com **múltiplos atributos** devem ser consideradas se a cláusula WHERE contiver diversas condições
 - ✓ A ordem dos atributos é importante em testes de intervalo
 - ✓ Este tipo de índice pode permitir planos de pesquisa que utilizam somente o índice (sem recuperar os registros de dados).
 - Para planos de consulta que envolvem somente o índice, a clusterização não é importante.
- Procure escolher os índices que beneficiem o maior número de consultas possível. Como somente um índice pode ser clusterizado, escolha-o baseado nas consultas mais importantes que se beneficiarão com essa organização



Exemplos de Índices Clusterizados

- ❖ Um índice com árvore B+ sobre *E.idade* pode ajudar.

```
SELECT E.numDep  
FROM Emp E  
WHERE E.idade>40
```

- Quão seletiva é a condição?
- O índice é clusterizado?

- ❖ Considere a consulta com GROUP BY.

```
SELECT E.numDep, COUNT (*)  
FROM Emp E  
WHERE E.idade>10  
GROUP BY E.numDep
```

- se houver muitas tuplas com *E.idade* > 10, usar o índice sobre *E.idade* pode não ser muito bom.
- índice clusterizado sobre *E.numDep* pode ser melhor.

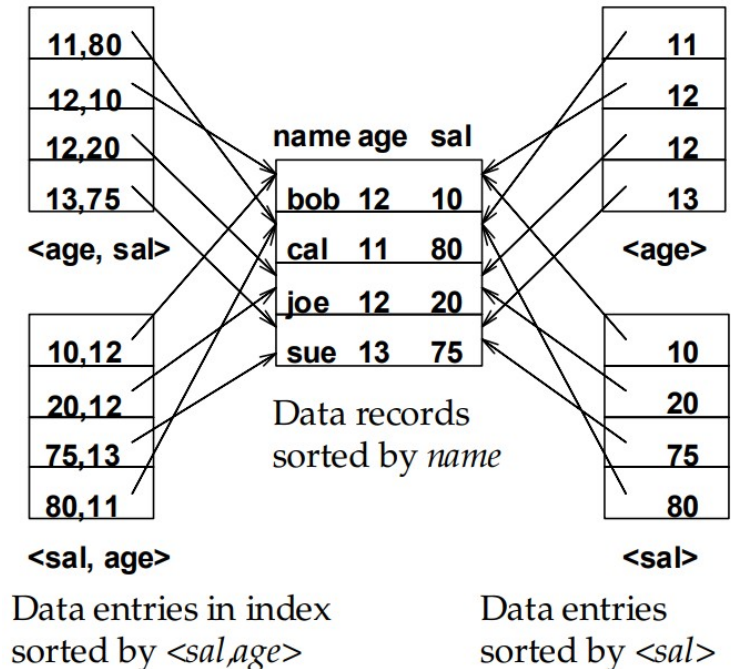
- ❖ Teste de igualdade e duplicações:

```
SELECT E.numDep  
FROM Emp E  
WHERE E.hobby="selos"
```

- Clusterização em *E.hobby* é bom.

Chaves de pesquisa compostas

- Para obter registros da tabela Emp com idade=30 e sal=4000, um índice sobre <idade,sal> ou <sal, idade> é melhor que um índice sobre idade ou sobre sal
- Se a condição é: $20 < \text{idade} < 30$ AND $3000 < \text{sal} < 5000$:
 - ✓ Um índice B+ clusterizado sobre <idade,sal> ou <sal,idade> é o melhor
- Se a condição é: $\text{idade} = 30$ AND $3000 < \text{sal} < 5000$:
 - ✓ Um índice clusterizado sobre <idade,sal> é muito melhor que um índice sobre <sal,idade>
- Índices compostos são maiores e precisam ser atualizados com mais frequência



Planos de Consulta somente com Índices

<E.numDep>

- ❖ Há consultas que podem ser respondidas sem que os registros de dados de uma ou mais relações sejam recuperadas se existirem índices apropriados.

```
SELECT E.numDep, COUNT(*)  
FROM Emp E  
GROUP BY E.numDep
```

<E.numDep, E.sal>

Índice B+

```
SELECT E.numDep, MIN(E.sal)  
FROM Emp E  
GROUP BY E.numDep
```

<E. idade, E.sal>

ou

<E.sal, E.idade>

Índice B+

```
SELECT AVG(E.sal)  
FROM Emp E  
WHERE E.age=25 AND  
E.sal BETWEEN 3000 AND 5000
```


Planos com índice

<E.numDep>

- ❖ Planos que envolvem somente o índice também existem para consultas que utilizam mais de uma relação.

```
SELECT D.gerente  
FROM Depto D, Emp E  
WHERE D.numDep=E.numDep
```

<E.numDep,E.idEmp>

```
SELECT D.gerente, E.idEmp  
FROM Depto D, Emp E  
WHERE D.numDep=E.numDep
```

Criação de Índices em SQL

- Sintaxe

- **CREATE INDEX** nomeIndice **ON** tabela(coluna)

- Exemplo

- **CREATE INDEX** idxNme **ON** Pessoa(nome)

Criação de Índices no PostgreSQL

- Sintaxe básica

```
CREATE [ UNIQUE ] INDEX nome-do-indice ON nome-da-tabela [ USING method ]  
( lista de atributos ) [ INCLUDE ( lista de atributos ) ] [ WHERE predicate ]
```

- **UNIQUE:** valores únicos na chave de pesquisa
- **Method:** método de indexação. Opções: btree, hash, gist, spgist, gin, brin, ou métodos de acesso instalados pelo usuário. Valor padrão: btree.
- **INCLUDE:** especifica uma lista de atributos que podem ser incluídos no índice como colunas não chave
 - Eles **não** podem ser utilizados para buscar um registro (index scan search)
 - Eles podem ser retornados em planos de consulta que envolvem apenas o índice (*index-only scan*) sem precisar visitar o arquivo de dados da tabela. Filtra pela chave de pesquisa e retorna os atributos do INCLUDE
- **Predicate:** expressão para criar um índice parcial, ou seja, com apenas algumas das linhas como chave de pesquisa

Exemplos

- Criando um índice B-tree único sobre o atributo **title** da tabela **films**:
 - `CREATE UNIQUE INDEX title_idx ON films (title);`
- Criando um índice B-tree único sobre o atributo **title** da tabela **films** incluindo as colunas **director** e **rating**:
 - `CREATE UNIQUE INDEX title_idx ON films (title) INCLUDE (director, rating);`
- Criando um índice B-tree sobre a expressão `lower(title)` para permitir buscas case-insensitive eficientes:
 - `CREATE INDEX title_lw ON films ((lower(title)));`
- Criando um índice **hash** sobre o atributo **title** da tabela **films**:
 - `CREATE INDEX title_idx ON films USING hash (title);`

Clusterização de Índices no PostgreSQL

- `CLUSTER nome-da-tabela USING nome-do-ndice`
 - Faz a clusterização dos registros de dados da tabela de acordo com a chave de pesquisa do índice
 - A clusterização só é feita nos registros atuais
 - Atualizações posteriores não obedecem a clusterização
- `CLUSTER nome-da-tabela`
 - Clusteriza reordena os registros da tabela de acordo com a clusterização já definida
- `CLUSTER`
 - Reordena todas as tabelas de acordo com a clusterização já definida

Resumo

- Existem diversas alternativas para organização de arquivos, cada uma apropriada para algumas situações.
- Se consultas com seleção são frequentes, é importante ordenar o arquivo ou criar um índice.
 - índices hash são bons para teste de igualdade.
 - arquivos ordenados e índices B+ são melhores para testes de intervalo; são também bons para teste de igualdade.
- Um índice consiste de uma coleção de entradas de dados que ajudam na localização de registros de dados que contém determinados valores para a chave de pesquisa.

Resumo

- As entradas de dados no índice podem conter:
 - os próprios registros
 - pares <chave, rid>, ou
 - pares <chave, lista-de-rids>
- A escolha da técnica de indexação é ortogonal ao formato das entradas de dados.
- Podem existir diversos índices para um mesmo arquivo com registros de dados (tabela), cada um com uma chave de pesquisa diferente.
- Os índices podem ser classificados como **primários** ou **secundários**, e podem ser **clusterizados** ou não

Resumo

- Entender a carga de trabalho do SGBD para uma aplicação é essencial para melhorar o seu desempenho.
 - Quais são as consultas e atualizações mais importantes? Quais os atributos e relações envolvidos?
- Os índices devem ser escolhidos para melhorar o desempenho de consultas.
 - Atualizações causam impacto na manutenção dos índices.
 - Os índices devem ser escolhidos para melhorar o desempenho de diversas consultas, se possível.
 - Criar índices que possibilitem planos de consultas que utilizam somente o índice.
 - A clusterização é extremamente importante; somente um índice por relação pode ser clusterizado.
 - A ordem dos atributos em chaves de pesquisa compostas pode ser importante